

# Práctica Compiladores

Sergio Ruiz Cejudo  
Iván Moreno de la Bandera  
Alejandro Masmiquel Casado

Universitat de les Illes Balears

---

## Índice

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                  | <b>1</b>  |
| <b>2. Características del Lenguaje Diseñado</b>         | <b>2</b>  |
| 2.1. Estructura . . . . .                               | 2         |
| 2.2. Declaración de variables . . . . .                 | 2         |
| 2.3. Declaración de funciones . . . . .                 | 2         |
| 2.4. Estructuras de control . . . . .                   | 2         |
| 2.5. Operadores . . . . .                               | 3         |
| <b>3. Análisis Léxico</b>                               | <b>4</b>  |
| 3.1. Tabla de Tókens . . . . .                          | 4         |
| 3.2. Patrones . . . . .                                 | 6         |
| 3.3. Rutinas . . . . .                                  | 6         |
| <b>4. Análisis Sintáctico y Semántico</b>               | <b>8</b>  |
| 4.1. Estructura Global . . . . .                        | 8         |
| 4.2. Declaración de Variables y Constantes . . . . .    | 8         |
| 4.3. Funciones . . . . .                                | 9         |
| 4.4. Sentencias de Control de Flujo . . . . .           | 10        |
| 4.5. Asignaciones . . . . .                             | 12        |
| 4.6. Expresiones . . . . .                              | 13        |
| <b>5. Tabla de Símbolos</b>                             | <b>15</b> |
| 5.1. Gestión de Ámbitos y Visibilidad . . . . .         | 15        |
| 5.2. Categorización de Símbolos . . . . .               | 15        |
| 5.3. Implementación de la Estructura de Datos . . . . . | 16        |
| <b>6. Optimizaciones</b>                                | <b>17</b> |
| 6.1. Eliminación de Código Muerto . . . . .             | 17        |
| 6.1.1. Descripción . . . . .                            | 17        |
| 6.1.2. Algoritmo . . . . .                              | 17        |
| 6.1.3. Ejemplo . . . . .                                | 17        |

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| 6.2.      | Propagación de Constantes . . . . .              | 17        |
| 6.2.1.    | Descripción . . . . .                            | 17        |
| 6.2.2.    | Algoritmo . . . . .                              | 17        |
| 6.2.3.    | Consideraciones Especiales . . . . .             | 18        |
| 6.2.4.    | Ejemplo . . . . .                                | 18        |
| 6.3.      | Eliminación de Subexpresiones Comunes . . . . .  | 18        |
| 6.3.1.    | Descripción . . . . .                            | 18        |
| 6.3.2.    | Algoritmo . . . . .                              | 18        |
| 6.3.3.    | Restricciones . . . . .                          | 19        |
| 6.3.4.    | Ejemplo . . . . .                                | 19        |
| 6.4.      | Orden de Aplicación . . . . .                    | 19        |
| <b>7.</b> | <b>Ensamblador</b>                               | <b>20</b> |
| 7.1.      | Gestión de Memoria . . . . .                     | 20        |
| 7.1.1.    | Pila (Stack) . . . . .                           | 20        |
| 7.1.2.    | Registros Utilizados . . . . .                   | 20        |
| 7.1.3.    | Organización del Stack Frame . . . . .           | 20        |
| 7.1.4.    | Prólogo y Epílogo de Funciones . . . . .         | 21        |
| 7.2.      | Variables Globales . . . . .                     | 21        |
| 7.3.      | Arrays . . . . .                                 | 21        |
| 7.4.      | Operaciones de Entrada/Salida . . . . .          | 22        |
| 7.4.1.    | Ejemplo: Imprimir un entero . . . . .            | 22        |
| 7.5.      | Operaciones Aritméticas y Relacionales . . . . . | 22        |
| 7.5.1.    | Operaciones Aritméticas . . . . .                | 22        |
| 7.5.2.    | Operaciones Relacionales . . . . .               | 22        |
| 7.6.      | Control de Flujo . . . . .                       | 22        |
| 7.7.      | Llamadas a Funciones . . . . .                   | 23        |

# 1. Introducción

Esta documentación detalla el desarrollo de un compilador para un lenguaje de programación imperativo cuya sintaxis y estructura están fuertemente inspiradas en Java. Para la implementación del front-end, hemos utilizado las herramientas JFlex en el análisis léxico y CUP para el análisis sintáctico, lo que ha permitido una gestión eficiente de los tokens y las reglas gramaticales. Además del flujo estándar de generación de código intermedio y ensamblador, el proyecto incorpora una fase de optimización de código orientada a mejorar el rendimiento del programa final en comparación con su versión base.

La realización de este trabajo ha sido fundamental para comprender de forma práctica todas las etapas de la arquitectura de un compilador y de la realización de un lenguaje.

## 2. Características del Lenguaje Diseñado

El lenguaje desarrollado para esta práctica es de tipo imperativo y estructurado, diseñado para cumplir con los requisitos fundamentales de la especificación.

### 2.1. Estructura

La estructura del lenguaje se basa en la declaración de variables y funciones, y en sentencias (instrucciones a ejecutar), acabando estos siempre en punto y coma (;), y el punto de entrada principal será un método llamado `main`.

### 2.2. Declaración de variables

En cuanto a la declaración de variables, esta consiste en indicar primero el tipo de variable  $\rightarrow$  (`entero`, `caracter`, `booleano` o `cadena`) seguido de su nombre. En el caso de los arrays, se indicará de la misma forma el tipo al principio, seguido de 2, 4 o 6 llaves (dependiendo de la dimensión del array, pudiendo ser de 1, 2 o 3 dimensiones) e indicando el tamaño del mismo dentro de estas  $\rightarrow$  (`[X]`). Además opcionalmente se podrá añadir al principio de cualquier tipo de variable que sean constantes (`constante`).

Existen 2 ámbitos de variables:

- **Ámbito local:** Se declaran dentro de funciones y no podrán ser usadas por otras funciones.
- **Ámbito global:** Se declaran fuera de las funciones, y son compartidas por todas estas.

### 2.3. Declaración de funciones

Por otra parte, la declaración de funciones consiste en indicar al principio el tipo de función (el cual determinará el tipo de valor que retornará)  $\rightarrow$  (`entero`, `caracter`, `booleano`, `cadena`, `array`), seguido del nombre y, entre paréntesis, los argumentos que se le pasarán por parámetro. También se pueden declarar funciones vacías (`vacio`) que no retornarán ningún valor. Todo el bloque de código de dicha función irá entre llaves (`{}`). Para llamar a las funciones, se hará poniendo el nombre de dicha función seguida de los argumentos entre paréntesis  $\rightarrow$  `nombre_funcion(arg1, arg2);`

### 2.4. Estructuras de control

Para el control de flujo disponemos de 3 estructuras diferentes:

- **Condicional:** Esta se hará con la estructura `si (condición) {bloque de código}` y, opcionalmente, se podrán añadir diversos `sino {bloque de código}`.
- **Bucle Mientras:** Esta se hará con la estructura `mientras (condición) {bloque de código}` hasta que se deje de cumplir la condición puesta.
- **Bucle Para:** Esta se hará con la estructura `para (variable; condición; incremento).`

### 2.5. Operadores

Existen 3 grupos de operadores:

- **Relacionales:** Son aquellos que nos permiten comparar valores  $\rightarrow$  ( $=$ ,  $==$ ,  $!=$ ,  $<$ ,  $>$ ,  $<=$ ,  $>=$ ).
- **Lógicos:** Son aquellos que nos permiten realizar las operaciones lógicas más básicas  $\rightarrow$  [ $\&\&$  (AND),  $\|\|$  (OR),  $!$  (NOT)].
- **Aritméticos:** Son los típicos: Suma (+), resta (-), multiplicación (\*) y división (/).

### 3. Análisis Léxico

Para la fase de análisis léxico, hemos desarrollado un escáner capaz de procesar el código fuente suministrado en un archivo de texto y transformarlo en una secuencia de tokens. Para ello, hemos utilizado **JFlex**, un generador de analizadores léxicos para Java. Esta herramienta nos permite no solo identificar los componentes léxicos, sino también gestionar el posicionamiento de cada elemento mediante el rastreo de líneas y columnas, facilitando así la comunicación con el gestor de errores en caso de detectar caracteres no válidos.

#### 3.1. Tabla de Tokens

A continuación se detallan de forma individualizada todos los tokens definidos en nuestro lenguaje, indicando el lexema reconocido y su funcionalidad:

Cuadro 1: Especificación detallada de tokens y lexemas asociados.

| Token                              | Lexema / Atributo | Descripción                                                    |
|------------------------------------|-------------------|----------------------------------------------------------------|
| ENTERO_TIPO                        | entero            | Palabra reservada para tipo de dato entero.                    |
| CADENA_TIPO                        | cadena            | Palabra reservada para tipo de dato cadena.                    |
| CONSTANTE                          | constante         | Palabra reservada para modificador constante, inmutable.       |
| BOOLEANO                           | booleano          | Palabra reservada para tipo de dato booleano.                  |
| CARACTER                           | caracter          | Palabra reservada para tipo de dato char.                      |
| VACIO_TIPO                         | vacio             | Palabra reservada para tipo de retorno vacío (void).           |
| VERDADERO                          | verdadero         | Literal booleano de valor verdadero.                           |
| FALSO                              | falso             | Literal booleano de valor falso.                               |
| SI                                 | si                | Palabra reservada para inicio de condicional.                  |
| SINO                               | sino              | Palabra reservada para alternativa condicional.                |
| MIENTRAS                           | mientras          | Palabra reservada para bucle de control <i>while</i> .         |
| PARA                               | para              | Palabra reservada para bucle de control iterativo <i>for</i> . |
| DEVOLVER                           | devolver          | Indica el retorno de valor en una función.                     |
| ESCRIBIR                           | escribir          | Operación de salida por pantalla.                              |
| LEER                               | leer              | Operación de entrada por teclado.                              |
| MAS                                | +                 | Operador aritmético de suma.                                   |
| MENOS                              | -                 | Operador aritmético de resta.                                  |
| Continúa en la siguiente página... |                   |                                                                |

Cuadro 1 – Continuación de la página anterior

| Token        | Lexema / Atributo | Descripción                                          |
|--------------|-------------------|------------------------------------------------------|
| POR          | *                 | Operador aritmético de multiplicación.               |
| DIV          | /                 | Operador aritmético de división.                     |
| ASIGNAR      | =                 | Operador de asignación de valor.                     |
| IGUAL        | ==                | Operador relacional de igualdad.                     |
| DIFERENTE    | !=                | Operador relacional de desigualdad.                  |
| MENOR        | <                 | Operador relacional de comparación menor.            |
| MAYOR        | >                 | Operador relacional de comparación mayor.            |
| MENORIGUAL   | <=                | Operador relacional menor o igual.                   |
| MAYORIGUAL   | >=                | Operador relacional mayor o igual.                   |
| AND          | &&                | Operador lógico de conjunción.                       |
| OR           |                   | Operador lógico de disyunción.                       |
| NOT          | !                 | Operador lógico de negación.                         |
| PAREN_IZQ    | (                 | Delimitador de apertura de parámetros y expresiones. |
| PAREN_DER    | )                 | Delimitador de cierre de parámetros y expresiones.   |
| LLAVE_IZQ    | {                 | Inicio de bloque de instrucciones del programa.      |
| LLAVE_DER    | }                 | Fin de bloque de instrucciones del programa.         |
| CORCHETE_IZQ | [                 | Delimitador de apertura de acceso a arreglos.        |
| CORCHETE_DER | ]                 | Delimitador de cierre de acceso a arreglos.          |
| PUNTO        | .                 | Operador de acceso a miembros o estructuras.         |
| PUNTO_COMA   | ;                 | Finalizador de sentencia.                            |
| COMA         | ,                 | Separador de elementos o parámetros.                 |
| NUMERO       | <i>Integer</i>    | Literal numérico entero constante.                   |
| CADENA       | <i>String</i>     | Literal de cadena de caracteres.                     |
| CARACTER_LIT | <i>Char</i>       | Literal de carácter constante (ej. 'a').             |
| ID           | <i>String</i>     | Identificador de variable o procedimiento.           |

### 3.2. Patrones

En esta subsección se definen los patrones utilizados por JFlex para el reconocimiento de los componentes léxicos más complejos, integrando las rutinas de detección necesarias:

| Nombre del Patrón | Patrón (Regex)                      | Descripción                                                               |
|-------------------|-------------------------------------|---------------------------------------------------------------------------|
| ENTERO            | <code>[0-9]+</code>                 | Una secuencia de uno o más dígitos numéricos.                             |
| IDENTIFICADOR     | <code>[a-zA-Z_][a-zA-Z0-9_]*</code> | Inicio por letra o guion bajo seguido de caracteres alfanuméricos.        |
| CADENA            | <code>"([^\\""] \\\.)*"</code>      | Texto entre comillas dobles con soporte para caracteres escapados.        |
| CHARACTER         | <code>'([^\''\\n] \\.)*'</code>     | Un carácter entre comillas simples con soporte para secuencias de escape. |
| ESPACIO           | <code>[\t\r\n]+</code>              | Espacios, tabuladores o saltos de línea que el escáner ignora.            |
| COMENTARIO        | <code>/*.*</code>                   | Comentarios de una sola línea que son omitidos por el compilador.         |

Cuadro 2: Patrones y expresiones regulares del analizador léxico.

### 3.3. Rutinas

Para la generación de símbolos (tokens) en el analizador léxico, se distinguen dos tipos de rutinas principales:

#### 1. Token SIN atributo (palabra clave):

Este caso se utiliza para palabras reservadas o delimitadores fijos.

```
1 "entero" { return symbol(sym. ENTERO_TIPO); }
2
```

**Explicación:** Se invoca al método `symbol(int type)`. Se crea un objeto `Symbol` inicializado únicamente con la estructura:

**tipo + línea + columna**

No es necesario almacenar un valor adicional, ya que el propio tipo del token (por ejemplo, `ENTERO_TIPO`) identifica unívocamente que se trata de esa palabra reservada específica.



### 2. Token CON atributo:

Este caso se aplica cuando el token transporta información variable.

```
1 {ENTERO}          { return symbol(sym.NUMERO, Integer.parseInt(yytext  
   ()); }  
2 {CADENA}          { return symbol(sym.CADENA, yytext()); }  
3 {CARACTER}        { return symbol(sym.CARACTER_LIT, yytext()); }  
4 {IDENTIFICADOR}   { return symbol(sym.ID, yytext()); }  
5
```

**Explicación:** Se invoca al método `symbol(int type, Object value)`. Se crea un objeto `Symbol` inicializado con la estructura:

**tipo + valor + línea + columna**

Los tokens que requieren atributo son:

- NUMERO: almacena el valor entero parseado.
- CADENA: almacena el texto literal entre comillas.
- CARACTER\_LIT: almacena el carácter entre comillas simples.
- ID: almacena el nombre del identificador.

## 4. Análisis Sintáctico y Semántico

Para la fase sintáctica se ha diseñado un parser basado en una gramática LALR. A continuación se detalla el procesamiento de las estructuras principales, desglosando la validación semántica y la generación de código intermedio para cada producción.

### 4.1. Estructura Global

```
programa ::= lista_declaraciones
lista_declaraciones ::= lista_declaraciones declaracion
                      | declaracion
declaracion ::= declaracion_funcion
              | declaracion_variable
```

El símbolo inicial del programa se define como una lista recursiva de declaraciones. A diferencia de otros lenguajes que exigen un procedimiento `main` al final, esta estructura permite intercalar declaraciones de variables globales y funciones en cualquier orden.

### 4.2. Declaración de Variables y Constantes

```
tipo ::= ENTERO_TIPO | CADENA_TIPO | CARACTER_TIPO |
      BOOLEANO_TIPO | VACIO_TIPO

declaracion_variable ::= tipo ID ;
                    | tipo ID = expresion ;
                    | CONSTANTE tipo ID = NUMERO ;
                    | tipo [ NUMERO ] ID ;
                    | tipo [ NUMERO ] ID = [ lista_valores ]
;
                    | tipo [ NUMERO ] [ NUMERO ] ID ;
                    | tipo [ NUMERO ] [ NUMERO ] ID = {
lista_valores_2d } ;
                    | tipo [ NUMERO ] [ NUMERO ] [ NUMERO ] ID
;
                    | tipo [ NUMERO ] [ NUMERO ] [ NUMERO ] ID
= { lista_valores_3d } ;

lista_valores ::= lista_valores , expresion
              | expresion

lista_valores_2d ::= [ lista_valores ]
                  | lista_valores_2d , [ lista_valores ]

lista_valores_3d ::= [ lista_valores_2d ]
                  | lista_valores_3d , [ lista_valores_2d ]
```

Esta producción administra la definición de tipos y la reserva de memoria para tres estructuras fundamentales: variables simples, que vinculan un identificador a un tipo; constantes, que imponen inmutabilidad exigiendo una inicialización obligatoria e inmediata; y arreglos de hasta tres dimensiones con tamaño estático, los cuales ofrecen flexibi-

lidad al permitir únicamente la reserva del bloque de memoria o su población instantánea mediante listas de valores anidadas.

| Validación                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Codificación                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>General:</b> Se verifica que el identificador (ID) no exista previamente en el ámbito actual (VAR_ALREADY_EXISTS).</p> <p><b>1. Constantes:</b> Se comprueba que el valor asignado sea un literal y coincida con el tipo declarado.</p> <p><b>2. Arrays:</b> Se valida que el tamaño sea un entero positivo. Si hay lista de valores, su longitud debe coincidir exactamente con el tamaño declarado. Para arrays multidimensionales, se valida cada nivel de anidación.</p> <p><b>3. Tipos:</b> Se asegura que la expresión asignada sea compatible con el tipo de la variable.</p> | <p><b>1. Variables:</b> Se emite una instrucción de reserva de espacio. Si hay asignación, se genera la instrucción COPY para mover el valor inmediato o el resultado de una expresión a la variable.</p> <p><b>2. Arrays:</b> Se calcula el tamaño total en bytes (Tamaño del Tipo <math>\times \prod</math> Dimensiones) y se genera una instrucción de reserva de bloque, asignando la dirección base al puntero del array.</p> |

Cuadro 3: Validación y codificación de variables

### 4.3. Funciones

```

declaracion_funcion ::= tipo ID ( parametros ) bloque
parametros ::= lista_parametros | ε
lista_parametros ::= lista_parametros , parametro
                  | parametro
parametro ::= tipo ID
bloque ::= { lista_sentencias }
         | { }

```

Esta producción establece la estructura para la modularización del programa, definiendo la firma de la función mediante un tipo de retorno, un identificador y una lista opcional de parámetros formales.

| Validación                                                                                                                                                                                                                                                                                                        | Codificación                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Se comprueba que el nombre de la función sea único. Se abre un nuevo ámbito y se insertan los parámetros en la Tabla de Símbolos local. Finalmente, se verifica que el bloque contenga al menos una sentencia DEVOLVER compatible con el tipo de retorno declarado (excepto para funciones de tipo vacío).</p> | <p>Se genera la etiqueta de inicio de la función. Se inserta el preámbulo estándar para la gestión de la pila (guardar punteros de marco). Al final, se asegura la generación de instrucciones de limpieza de pila y retorno (RET).</p> |

Cuadro 4: Validación y codificación de funciones

#### 4.4. Sentencias de Control de Flujo

Todas las sentencias implementan una interfaz común que permite su procesamiento secuencial dentro de una `lista_sentencias`.

```
lista_sentencias ::= lista_sentencias sentencia
                  | sentencia
sentencia ::= sentencia_asignacion | sentencia_si |
              sentencia_mientras
              | sentencia_para | sentencia_devolver |
              sentencia_escribir
              | sentencia_leer | sentencia_llamada |
              declaracion_local | bloque
```

##### Sentencia Si (Condicional)

```
sentencia_si ::= SI ( expresion ) sentencia
              | SI ( expresion ) sentencia SINO sentencia
```

Genera una estructura de bifurcación basada en una condición booleana.

| Validación                                                                                                                                                                             | Codificación                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Se verifica estrictamente que la <code>expresion</code> condicional resulte en un tipo <code>BOOLEANO</code> . Si es de otro tipo, se lanza el error <code>BAD_CONDITION_TYPE</code> . | <ol style="list-style-type: none"> <li>1. Se generan etiquetas para el bloque <i>else</i> (<code>L_ELSE</code>) y el final (<code>L_END</code>).</li> <li>2. Se codifica la expresión y un salto condicional: <code>IF_FALSE_GOTO L_ELSE</code>.</li> <li>3. Se codifica el bloque <i>then</i> seguido de un <code>GOTO L_END</code>.</li> <li>4. Se inserta <code>L_ELSE</code>, se codifica el bloque <i>else</i> y se cierra con <code>L_END</code>.</li> </ol> |

##### Sentencia Para (Bucle For)

```
sentencia_para ::= PARA ( asignacion_simple ; expresion ;
                        asignacion_simple ) sentencia
```

Estructura iterativa compacta que incluye inicialización, condición y paso en la cabecera.

| Validación                                                                                                                                                       | Codificación                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Valida que la expresión central sea <code>BOOLEANO</code> . Verifica que las asignaciones (inicial y de paso) sean semánticamente correctas (tipos compatibles). | <ol style="list-style-type: none"> <li>1. Se codifica la asignación de inicialización.</li> <li>2. Se coloca la etiqueta <code>L_START</code>.</li> <li>3. Se evalúa la condición; si es falsa, salto a <code>L_EXIT</code>.</li> <li>4. Se codifica el cuerpo de la sentencia.</li> <li>5. Se codifica la asignación de paso (incremento) y se salta a <code>L_START</code>.</li> <li>6. Se coloca la etiqueta <code>L_EXIT</code>.</li> </ol> |

### Sentencia Mientras (Bucle While)

```
sentencia_mientras ::= MIENTRAS ( expresion ) sentencia
```

| Validación                                                                | Codificación                                                                                                                                                              |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Se valida que la expresión entre paréntesis sea de tipo <b>BOOLEANO</b> . | Similar al bucle <b>PARA</b> pero sin los pasos de inicialización y actualización explícitos en la cabecera. Se usan etiquetas de inicio y fin para el control del ciclo. |

### Sentencia Devolver

```
sentencia_devolver ::= DEVOLVER expresion ;  
                    | DEVOLVER ;
```

| Validación                                                                                                                                                                                                                            | Codificación                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Se verifica que el tipo de la expresión retornada coincida con el tipo declarado de la función. Para funciones de tipo <b>vacio</b> , no debe haber expresión de retorno. Si una función no-void omite la expresión, se genera error. | Se genera la instrucción <b>return</b> con el valor de la expresión (si existe). Para funciones <b>vacio</b> , se genera <b>return</b> sin valor. |

## 4.5. Asignaciones

```

sentencia_asignacion ::= ID = expresion ;
                      | ID [ expresion ] = expresion ;
                      | ID [ expresion ][ expresion ] =
                        expresion ;
                      | ID [ expresion ][ expresion ][
                        expresion ] = expresion ;

asignacion_simple ::= ID = expresion
                   | ID [ expresion ] = expresion
                   | ID [ expresion ][ expresion ] = expresion
                   | ID [ expresion ][ expresion ][ expresion ]
                     = expresion

```

| Validación                                                                                                                                                                                                                                                                                | Codificación                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Se comprueba que ID esté declarado y no sea una constante (IS_CONSTANT). Se verifica la compatibilidad de tipos entre izquierda y derecha. Para arrays, todos los índices deben ser de tipo ENTERO. Se valida que el número de índices coincida con las dimensiones declaradas del array. | <p><b>Simple:</b> Genera instrucción COPY dest, src.</p> <p><b>Array 1D:</b> Calcula <math>\text{offset} = \text{índice} * \text{tamaño\_tipo}</math>.</p> <p><b>Array 2D:</b> Calcula <math>\text{offset} = (i * \text{dim2} + j) * \text{tamaño\_tipo}</math>.</p> <p><b>Array 3D:</b> Calcula <math>\text{offset} = ((i * \text{dim2} + j) * \text{dim3} + k) * \text{tamaño\_tipo}</math>.</p> <p>Se genera instrucción de almacenamiento indexado.</p> |

## 4.6. Expresiones

```

expresion ::= expresion_logica
expresion_logica ::= expresion_relacional
                  | expresion_logica AND expresion_relacional
                  | expresion_logica OR expresion_relacional
expresion_relacional ::= expresion_aditiva
                     | expresion_relacional ==
                     | expresion_relacional !=
                     | expresion_relacional <
                     | expresion_relacional >
                     | expresion_relacional <=
                     | expresion_relacional >=
                     | expresion_aditiva
expresion_aditiva ::= expresion_multiplicativa
                  | expresion_aditiva +
                  | expresion_aditiva -
                  | expresion_aditiva *
                  | expresion_aditiva /
                  | expresion_primaria
expresion_multiplicativa ::= expresion_primaria
                        | expresion_multiplicativa *
                        | expresion_multiplicativa /
                        | expresion_primaria
expresion_primaria ::= NUMERO | CADENA | CARACTER_LIT |
                     VERDADERO | FALSO
                     | llamada_funcion | ID
                     | ID [ expresion ]
                     | ID [ expresion ][ expresion ]
                     | ID [ expresion ][ expresion ][ expresion
]
                     | ( expresion )
                     | NOT expresion_primaria

llamada_funcion ::= ID ( lista_expresiones ) | ID ( )
lista_expresiones ::= lista_expresiones , expresion | expresion

```

La jerarquía de la gramática (Lógica < Relacional < Aditiva < Multiplicativa) resuelve implícitamente la precedencia de operadores.

| Validación                                                                                                                                                                                                                                                                                                                                                                                                                                   | Codificación                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Operaciones:</b> Se comprueba la compatibilidad de operandos para cada operador (ej. + requiere ENTERO o CADENA; AND requiere BOOLEANO).</p> <p><b>Llamadas:</b> Se valida que los argumentos pasados coincidan en orden, cantidad y tipo con la definición de la función.</p> <p><b>Acceso a arrays:</b> Se verifica que el número de índices coincida con las dimensiones del array y que todos los índices sean de tipo ENTERO.</p> | <p><b>Operaciones:</b> Se utiliza un modelo de variables temporales. Se codifican los operandos recursivamente en temporales (t1, t2) y se emite la operación (ADD t1, t2, t3).</p> <p><b>Llamadas:</b> Se generan instrucciones PARAM para apilar cada argumento y finalmente la instrucción CALL.</p> <p><b>Acceso a arrays:</b> Se calcula el offset linealizado según el número de dimensiones.</p> |

Cuadro 5: Gestión de expresiones y operaciones



## 5. Tabla de Símbolos

El diseño implementado estructura el análisis semántico en dos niveles jerárquicos de abstracción: un contexto global para subprogramas y declaraciones compartidas, y contextos locales para el cuerpo de las funciones.

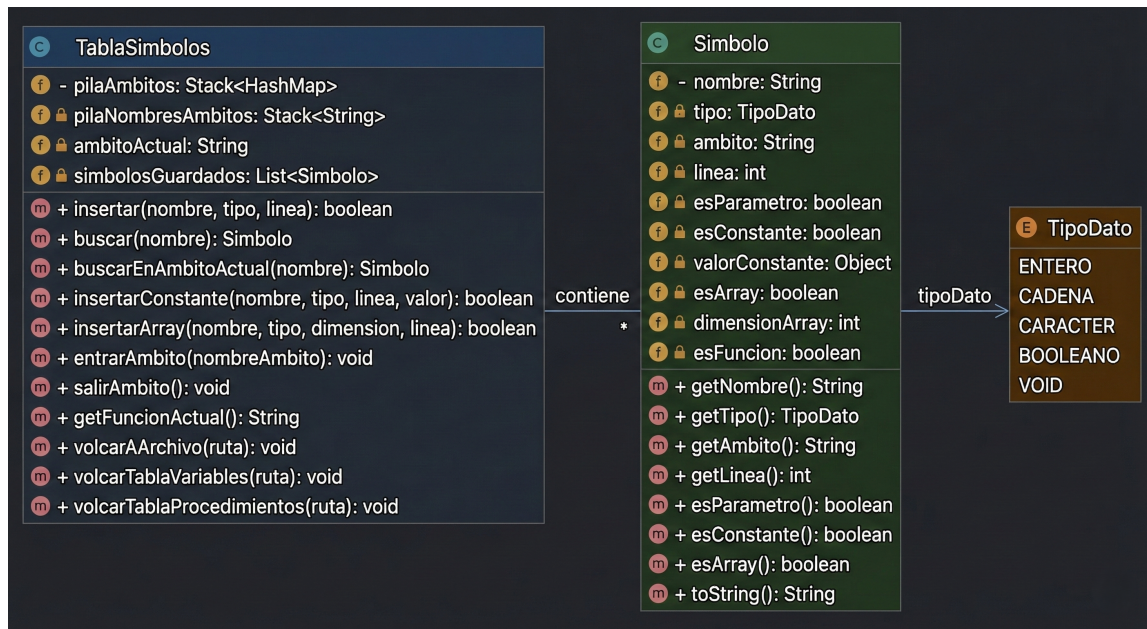


Figura 1: Diagrama de clases del analizador semántico: TablaSimbolos, Simbolo y TipoDato.

### 5.1. Gestión de Ámbitos y Visibilidad

El compilador gestiona la visibilidad de los identificadores mediante un sistema de ámbitos dinámicos basados en la estructura del código fuente:

- **Ámbito Global:** El ámbito global se inicializa al comienzo del análisis y perdura durante toda la ejecución del compilador, almacenando las definiciones de funciones y variables globales.
- **Ámbito Local:** Al ingresar en la definición de una función, el sistema transiciona a un estado `LOCAL`. Este ámbito es efímero: se instancia al entrar en el bloque de la función y se destruye al salir del mismo.

Este mecanismo garantiza que las declaraciones locales oculten a las globales homónimas durante la ejecución del subprograma y libera los recursos asociados al finalizar el bloque, restaurando el estado `GLOBAL` mediante la actualización de la variable de control `ambito`.

### 5.2. Categorización de Símbolos

Para garantizar la integridad semántica, cada entrada en la tabla de símbolos encapsula un identificador único asociado a una categoría específica, lo que permite al compilador realizar inferencias de tipos y comprobaciones de seguridad:

1. **Variables (Type):** Representan espacios de memoria mutables. Se almacena su tipo de dato primitivo asociadas (**entero**, **cadena**, **caracter**, **booleano**).
2. **Constantes (Literal):** Representan valores inmutables. Además del tipo, se almacena el valor literal exacto para optimizaciones posteriores (ej. *Constant Folding*).
3. **Firmas de Función (Signature):** Almacenan la estructura de los subprogramas, incluyendo el tipo de retorno y la aridad de sus parámetros, esencial para validar las llamadas a funciones.

### 5.3. Implementación de la Estructura de Datos

A nivel de implementación, la tabla de símbolos se ha modelado utilizando una estructura de datos compuesta que optimiza tanto la búsqueda como la gestión de la profundidad de los ámbitos:

`Stack < HashMap < String, Símbolo ~`

- **Pila (Stack):** Se utiliza una estructura LIFO para gestionar la jerarquía de ámbitos. La base de la pila corresponde siempre al ámbito global, mientras que la cima representa el ámbito local activo actual.
- **Mapas Hash:** Cada nivel de la pila contiene un mapa asociativo que permite la inserción y búsqueda de identificadores con una complejidad temporal promedio de  $O(1)$ .

Esta arquitectura permite que la búsqueda de un identificador comience en el ámbito actual (cima de la pila) y, en caso de fallo, continúe en el ámbito global, respetando así las reglas de precedencia léxica del lenguaje.

## 6. Optimizaciones

Implementamos un módulo de optimización de código intermedio que aplica tres técnicas clásicas de optimización. Estas optimizaciones se ejecutan de forma secuencial sobre el código intermedio generado.

### 6.1. Eliminación de Código Muerto

#### 6.1.1. Descripción

Esta optimización detecta código inalcanzable, es decir, instrucciones que aparecen después de un salto incondicional (`goto`) y antes de la siguiente etiqueta referenciada.

#### 6.1.2. Algoritmo

El algoritmo se implementa en dos pasadas:

1. **Primera pasada:** Se identifican todas las etiquetas que son referenciadas por instrucciones `goto` o `if`.
2. **Segunda pasada:** Se recorre el código y se elimina cualquier instrucción que aparezca después de un `goto` hasta encontrar una etiqueta válida.

#### 6.1.3. Ejemplo

| Código Original                                                                                              | Código Optimizado                                               |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <code>goto L1</code><br><code>x = 5</code><br><code>y = 10</code><br><code>L1:</code><br><code>z = 20</code> | <code>goto L1</code><br><code>L1:</code><br><code>z = 20</code> |

Cuadro 6: Ejemplo de eliminación de código muerto

Las instrucciones `x = 5` e `y = 10` son eliminadas porque nunca se ejecutarán debido al salto incondicional previo.

### 6.2. Propagación de Constantes

#### 6.2.1. Descripción

Cuando una variable tiene asignado un valor constante, todas las referencias posteriores a esa variable pueden ser reemplazadas directamente por el valor, eliminando la necesidad de acceder a memoria.

#### 6.2.2. Algoritmo

1. Se mantiene un mapa de constantes conocidas: `constantes : Variable → Valor`
2. Para cada instrucción de asignación `x = c` donde `c` es una constante numérica:

- Se añade  $(x, c)$  al mapa de constantes
- 3. Para cada uso de una variable  $v$ :
  - Si  $v \in \text{constantes}$ , se reemplaza  $v$  por  $\text{constantes}[v]$
- 4. El mapa se invalida en:
  - Etiquetas de bucle (posibles redefiniciones)
  - Llamadas a funciones (posibles efectos secundarios)
  - Inicio de nuevas funciones

### 6.2.3. Consideraciones Especiales

- Se distingue entre **constantes globales** (declaradas fuera de funciones) y **constantes locales**.
- Las **variables inicializadas** no se propagan, ya que su valor puede cambiar durante la ejecución.
- Las constantes globales se preservan a través de llamadas a funciones.

### 6.2.4. Ejemplo

| Código Original | Código Optimizado |
|-----------------|-------------------|
| PI = 3          | PI = 3            |
| radio = 5       | radio = 5         |
| t1 = PI * radio | t1 = 3 * 5        |
| t2 = t1 * radio | t2 = t1 * 5       |

Cuadro 7: Ejemplo de propagación de constantes

## 6.3. Eliminación de Subexpresiones Comunes

La **eliminación de subexpresiones comunes** identifica expresiones que se calculan más de una vez y reutiliza el resultado previamente calculado.

### 6.3.1. Descripción

Si una expresión  $E$  se calcula y su resultado se almacena en una variable temporal  $t$ , cualquier cálculo posterior de la misma expresión  $E$  puede ser reemplazado por una referencia a  $t$ , evitando recalcular el valor.

### 6.3.2. Algoritmo

1. Se mantiene un mapa de expresiones calculadas:  $\text{expresiones} : \text{Expresión} \rightarrow \text{Variable}$
2. Para cada instrucción  $x = E$ :
  - Si  $E \in \text{expresiones}$ : reemplazar por  $x = \text{expresiones}[E]$
  - Si  $E \notin \text{expresiones}$  y  $E$  no es un literal: añadir  $(E, x)$  al mapa

3. El mapa se limpia al encontrar:

- Etiquetas (puntos de control de flujo)
- Llamadas a funciones

### 6.3.3. Restricciones

Las asignaciones literales ( $x = 0$ ,  $y = \text{"texto"}$ ) no se incluyen en el mapa, ya que no representan cálculos reutilizables.

### 6.3.4. Ejemplo

| Código Original | Código Optimizado |
|-----------------|-------------------|
| t1 = a + b      | t1 = a + b        |
| t2 = a + b      | t2 = t1           |
| t3 = t1 * t2    | t3 = t1 * t1      |

Cuadro 8: Ejemplo de eliminación de subexpresiones comunes

## 6.4. Orden de Aplicación

Las optimizaciones se aplican en el siguiente orden:

1. **Eliminación de código muerto:** Reduce el tamaño del código eliminando instrucciones inalcanzables.
2. **Propagación de constantes:** Simplifica expresiones reemplazando variables por valores conocidos.
3. **Eliminación de subexpresiones comunes:** Reutiliza cálculos previos para evitar redundancia.

## 7. Ensamblador

Hemos decidido implementar la generación de código para la arquitectura **MIPS**, una arquitectura RISC. Esta elección nos ofrece diversas ventajas, como la simplicidad de las instrucciones, la disponibilidad de simuladores como **MARS**, y el soporte nativo de llamadas al sistema para operaciones de entrada/salida.

### 7.1. Gestión de Memoria

#### 7.1.1. Pila (Stack)

La gestión de memoria se realiza mediante la pila. Para cada función se reserva un marco de pila (*stack frame*) de **1024 bytes**, que permite almacenar:

- Variables locales
- Arrays locales
- Parámetros de la función
- Dirección de retorno (\$ra)
- Frame pointer anterior (\$fp)

#### 7.1.2. Registros Utilizados

| Registro  | Uso                                           |
|-----------|-----------------------------------------------|
| \$fp      | Frame pointer (base del marco de pila actual) |
| \$sp      | Stack pointer (tope de la pila)               |
| \$ra      | Return address (dirección de retorno)         |
| \$a0-\$a3 | Parámetros de funciones (hasta 4)             |
| \$v0      | Valor de retorno y código de syscall          |
| \$t0-\$t9 | Registros temporales para cálculos            |

Cuadro 9: Registros MIPS utilizados

#### 7.1.3. Organización del Stack Frame

El marco de pila de cada función se organiza de la siguiente manera:

| Offset desde \$fp | Contenido                   |
|-------------------|-----------------------------|
| 508(\$fp)         | Dirección de retorno (\$ra) |
| 504(\$fp)         | Frame pointer anterior      |
| 500(\$fp)         | Primer parámetro            |
| 496(\$fp)         | Segundo parámetro           |
| 492(\$fp)         | Tercer parámetro            |
| 488(\$fp)         | Cuarto parámetro            |
| 0-487(\$fp)       | Variables locales y arrays  |

Cuadro 10: Organización del stack frame

### 7.1.4. Prólogo y Epílogo de Funciones

Al entrar en una función, se ejecuta el siguiente **prólogo**:

```
addi $sp, $sp, -1024 # Reservar 1024 bytes
sw $ra, 508($sp)     # Guardar return address
sw $fp, 504($sp)     # Guardar frame pointer anterior
move $fp, $sp        # Nuevo frame pointer
sw $a0, 500($fp)     # Guardar parámetros
sw $a1, 496($fp)
sw $a2, 492($fp)
sw $a3, 488($fp)
```

Al salir de una función, se ejecuta el **epílogo**:

```
lw $ra, 508($fp)     # Restaurar return address
lw $fp, 504($fp)     # Restaurar frame pointer
addi $sp, $sp, 1024  # Liberar stack frame
jr $ra               # Retornar
```

## 7.2. Variables Globales

Las variables globales se declaran en la sección `.data` del programa ensamblador:

```
.data
newline: .asciiz "\n"
buffer: .space 256
miVariable: .word 0
```

El acceso a variables globales se realiza mediante direccionamiento absoluto:

```
la $t9, miVariable   # Cargar dirección
lw $t0, 0($t9)       # Leer valor
sw $t0, 0($t9)       # Escribir valor
```

## 7.3. Arrays

Los arrays se almacenan en el stack frame de la función donde se declaran. El acceso a elementos se calcula mediante:

$$\text{offset\_elemento} = \text{offset\_base} - (\text{índice} \times 4)$$

Para índices dinámicos (variables), el cálculo se realiza en tiempo de ejecución:

```
li $t9, 4             # Tamaño de palabra
mul $t8, $t8, $t9     # índice * 4
li $t9, offset_base
sub $t8, $t9, $t8      # offset_base - (índice * 4)
add $t8, $t8, $fp      # Dirección absoluta
lw $t0, 0($t8)        # Cargar elemento
```

## 7.4. Operaciones de Entrada/Salida

Las operaciones de E/S se implementan mediante *syscalls* de MIPS:

| Syscall (\$v0) | Operación         | Argumentos                   |
|----------------|-------------------|------------------------------|
| 1              | Imprimir entero   | \$a0 = valor                 |
| 4              | Imprimir cadena   | \$a0 = dirección             |
| 5              | Leer entero       | Resultado en \$v0            |
| 8              | Leer cadena       | \$a0 = buffer, \$a1 = tamaño |
| 11             | Imprimir carácter | \$a0 = código ASCII          |
| 12             | Leer carácter     | Resultado en \$v0            |

Cuadro 11: Syscalls utilizados para E/S

### 7.4.1. Ejemplo: Imprimir un entero

```
lw $a0, offset($fp)    # Cargar valor a imprimir
li $v0, 1               # Syscall: print integer
syscall
la $a0, newline         # Imprimir salto de línea
li $v0, 4
syscall
```

## 7.5. Operaciones Aritméticas y Relacionales

### 7.5.1. Operaciones Aritméticas

| Operación      | Instrucción MIPS          |
|----------------|---------------------------|
| Suma           | add \$t2, \$t0, \$t1      |
| Resta          | sub \$t2, \$t0, \$t1      |
| Multiplicación | mul \$t2, \$t0, \$t1      |
| División       | div \$t0, \$t1; mflo \$t2 |

Cuadro 12: Traducción de operaciones aritméticas

### 7.5.2. Operaciones Relacionales

| Operación  | Implementación                                |
|------------|-----------------------------------------------|
| $a < b$    | slt \$t2, \$t0, \$t1                          |
| $a > b$    | slt \$t2, \$t1, \$t0                          |
| $a \leq b$ | slt \$t2, \$t1, \$t0; xori \$t2, \$t2, 1      |
| $a \geq b$ | slt \$t2, \$t0, \$t1; xori \$t2, \$t2, 1      |
| $a == b$   | xor \$t2, \$t0, \$t1; sltiu \$t2, \$t2, 1     |
| $a \neq b$ | xor \$t2, \$t0, \$t1; sltu \$t2, \$zero, \$t2 |

Cuadro 13: Traducción de operaciones relacionales

## 7.6. Control de Flujo

Los saltos condicionales se traducen de la siguiente manera:



- `ifFalse x goto L`: Se traduce a `beqz $t0, L` (salta si es cero/falso)
- `if x goto L`: Se traduce a `bnez $t0, L` (salta si no es cero/verdadero)
- `goto L`: Se traduce a `j L` (salto incondicional)

### 7.7. Llamadas a Funciones

Las llamadas a funciones siguen la convención de MIPS:

1. Guardar registros temporales en la pila
2. Cargar parámetros en `$a0-$a3`
3. Ejecutar `jal función`
4. Restaurar registros temporales
5. El resultado queda en `$v0`

```
# Guardar temporales
addi $sp, $sp, -12
sw $t0, 0($sp)
sw $t1, 4($sp)
sw $t2, 8($sp)
# Pasar parámetros
move $a0, $t3
# Llamar función
jal miFuncion
# Restaurar temporales
lw $t0, 0($sp)
lw $t1, 4($sp)
lw $t2, 8($sp)
addi $sp, $sp, 12
```